Basys3 FPGA Resources

What BRAM, DSP, LUT and FF are, where the numbers come from, and how our Lane Centering Assist (LCA) Tiny CNN uses each one.

> **Each fact shows how we know it.** Tests were run on 2026-11-02 for this page. The LUT and FF counts cover the code on the team's branches that day, and they'll change as the RTL changes. `conv_engine` didn't exist yet, so the full design is still an estimate.

Label	Meaning
Manual p.2	Digilent Basys 3 Reference Manual, page number of the PDF from Digilent
DS181 p.35	AMD Artix 7 Data Sheet DS181 v1.27.1, from AMD
Vivado part	Read from Vivado 2026.1 with `report_property [get_parts xc7a35tcpg236-1]`
Measured	Tested with Vivado synthesis or simulation on our exact part
Background	General knowledge of the device, not checked against a file we have. AMD UG473 (memory), UG474 (slices) and UG479 (DSP) are the sources to confirm it

Contents

1. Spec and evidence
2. The four resources
3. LUT
4. FF
5. BRAM
6. DSP
7. Choosing a resource
8. Our 1-layer design
9. Where resources go
10. How to calculate it
11. Final budget
12. What if we change it?
13. Try it yourself
14. Cheat sheet

---

1. The Basys3 spec and its evidence

Board and chip are different things

Basys3 is the circuit board. It holds the clock oscillator, switches, LEDs, display and connectors. The FPGA is one chip on it, the **XC7A35T-1CPG236C** Manual p.1. So some facts belong to the chip, and others to the board.

Part of the name	Meaning	How we know
`XC7A`	Artix 7 family	Vivado part
`35T`	The device size	Vivado part
`-1`	Speed grade. Use the -1 column in DS181	Vivado part
`CPG236`	Package with 236 pins, 106 usable as I/O	Vivado part

FPGA resources

Resource	Amount	Evidence	How it's obtained
Logic cells	33,280	Manual p.2	Read directly
Slices	5,200	Manual p.2	Read directly. See the note below
LUTs	20,800	Manual p.2 Vivado part	5,200 × 4 LUTs per slice
Flip-flops	41,600	Manual p.2 Vivado part	5,200 × 8 FFs per slice
Block RAM	1,800 Kbits	Manual p.2	Read directly
BRAM36 blocks	50	Vivado part	1,800 ÷ 36 Kbits = 50
DSP slices	90	Manual p.2 Vivado part	Read directly

> **Why Vivado shows 8,150 slices.** Vivado's `SLICES` property and utilization report both say 8,150, not 5,200. I can't explain this from our documents. It doesn't affect your budget, because Vivado's LUT and FF limits (20,800 and 41,600) match the manual exactly. Budget with LUTs and FFs, not slices.

Board facts

Fact	Value	Evidence
Clock	100 MHz on pin W5	Manual p.7
Switches, LEDs, buttons	16, 16, 5	Manual p.2 Manual p.3
7-segment display	4 digits	Manual p.3
VGA	4 bits per colour (RGB444)	Manual p.9
USB-UART	FT2232HQ. FPGA receives on B18, transmits on A18	Manual p.7

Speed limits

DS181 lists how fast each block can run. Read the **-1** column, because that's our speed grade.

Block	Max frequency	Evidence
BRAM, write-first or no-change mode	388.20 MHz	DS181 p.31
BRAM, read-first delayed write	339.67 MHz	DS181 p.31
DSP, all registers used	464.25 MHz	DS181 p.35
DSP, without MREG	257.47 MHz	DS181 p.35
Global clock tree (BUFG)	464.00 MHz	DS181 p.36

All of these are well above 100 MHz, so these blocks aren't what limits your clock. The limit is the logic and routing between them, which is why you read WNS from the timing report.

Check it yourself

Run this in the Vivado Tcl console:

```
report_property [get_parts xc7a35tcpg236-1]
```

Look for `BLOCK_RAMS`, `DSP`, `LUT_ELEMENTS`, `FLIPFLOPS` and `SPEED`.

---

2. The four resources at a glance

An FPGA is a grid of small building blocks that you wire together by programming. Four of them matter most for this project:

LUT
Does logic: any small true/false function.
20,800 available

FF
Remembers one bit until the next clock edge.
41,600 available

BRAM
Stores lots of data: images, weights, buffers.
50 blocks available

DSP
Multiplies and adds quickly.
90 available

LUTs and FFs live together in **slices**. Each slice has 4 LUTs and 8 FFs Manual p.2. BRAM and DSP are separate hard blocks, which means their circuits are already built into the chip. They're faster and smaller than building the same thing from LUTs, but you can only use them in the ways they were designed for.

3. LUT: the logic block

A LUT (look-up table) is a tiny memory that acts as logic. Instead of building gates, it stores the answer for every possible input. A 6-input LUT has $2^6 = 64$ possible inputs, so it stores 64 bits. The Basys3 has 6-input LUTs Manual p.2 .

A 2-input AND gate stored in a LUT looks like this:

a	b	Stored output
0	0	0
0	1	0
1	0	0
1	1	1

The bitstream fills in that table. Change the stored bits and the same LUT becomes OR, XOR, or any function of up to 6 inputs. A larger function needs several LUTs.

What LUTs cost in practice

Circuit (like the one in our design)	LUTs	FFs	Evidence
Max of two 8-bit values (pool)	8	24	Measured
ReLU: round, shift, clamp	16	25	Measured
Argmax of three 32-bit logits	50	0	Measured
16-bit counter	1	16	Measured

> **Why does a 16-bit counter need only 1 LUT?** Slices have dedicated carry logic for adding, and it isn't counted as LUTs Background . So adders and counters are cheap. Comparing 32-bit values with lots of muxing costs more.

LUTs can also be memory

Some LUTs can store your own data. This is called **LUT RAM** or distributed RAM, and it's good for tiny memories. Our report shows 9,600 LUTs can do this Vivado report .

Memory	Built as	LUTs	BRAM	Evidence
`conv_w_rom`, 36 × 8	LUT ROM	8	0	Measured
`conv_w_rom`, 36 × 8	BRAM	0	1 RAMB18	Measured
`pool_ram`, 420 × 8	LUT RAM	107	0	Measured
`pool_ram`, 420 × 8	BRAM	0	1 RAMB18	Measured

---

4. FF: the memory for one bit

A flip-flop stores one bit. On each rising clock edge it copies its input to its output, then holds that value until the next edge. Every `reg` assigned in an `always @(posedge clk)` block normally becomes FFs, one per bit.

```
reg [15:0] cycle_count;   // 16 FFs
reg [2:0]  state_q;       // 3 FFs
reg signed [31:0] logit;  // 32 FFs
```

What FFs are for

- **State:** FSM states, loop counters and addresses.
- **Results you must keep:** logits and the class ID.
- **Pipelining:** cutting a long path into shorter ones so it fits in the clock period.

Why pipelining matters

At 100 MHz the clock period is 10 ns. A signal must leave one FF, go through the logic, and reach the next FF within that time. If it can't, timing fails and WNS goes negative. Adding an FF in the middle splits the path in two, at the cost of one extra cycle of latency.

Why not use FFs for all memory?

Because they're the wrong tool for bulk storage. `fmap_ram` holds 1,680 bytes:

```
1,680 × 8 = 13,440 bits = 13,440 FFs = 32% of the chip
Same data in BRAM = 1 RAMB18 = 1% of the BRAM
```

---

5. BRAM: the data storage

BRAM (block RAM) is a block of real memory inside the FPGA. It holds your images, weights and layer buffers.

RAMB36 and RAMB18

The chip has 50 blocks of 36 Kbits. Each one can also be split into two independent 18-Kbit halves Background.

Block	Kbits	8-bit depth	Usable bytes	How many
RAMB36	36	4,096	4,096	50
RAMB18	18	2,048	2,048	100

A RAMB18 holds 18 Kbits, which is 2,250 bytes, but at 8-bit width only 2,048 are usable. The other bits are parity bits, used only by 9-bit-wide data Background. That's why the board's roughly 225 KB of BRAM gives about 200 KB for 8-bit data.

We measured the size rule

The 2,048 limit isn't in our two PDFs, so we tested it. All memories here are 8 bits wide, on our exact part:

Depth × width	What Vivado used	Evidence
36 × 8	1 RAMB18	Measured
420 × 8	1 RAMB18	Measured
1,260 × 8	1 RAMB18	Measured
1,680 × 8	1 RAMB18	Measured
2,048 × 8	**1 RAMB18**	Measured
2,049 × 8	**1 RAMB36**	Measured
8,192 × 8	2 RAMB36	Measured

> **Two lessons.** One extra byte past 2,048 doubles the cost, from half a block to a full one. And a memory uses a whole block however little of it you fill: 36 bytes takes the same RAMB18 as 2,048 bytes.

Reads take 1 clock cycle

You give the address, and the data arrives after the next clock edge. We tested the team's `rom.v` in simulation by changing the address: before the edge the output still showed the old data, and after one edge it showed the new data Measured.

```
cycle 0:  put address on the port
cycle 1:  data is ready
```

This is the most common bug in the project. An address generator that forgets it reads every value one place out of step.

Other things to know

- **Two ports:** a BRAM can serve two accesses per cycle `Background`. Reading 9 values per cycle would need about 5 blocks, even if the data would fit in one.
- **Starts with your data:** `$readmemh` values go into the bitstream, so ROMs are filled when the board is programmed. This is how your images and weights arrive.
- **In reports:** "Block RAM Tile" counts in BRAM36 units, so one RAMB18 shows as 0.5.

---

6. DSP: the multiplier

A DSP48E1 is a hard block built for multiply and add. The chip has 90 `Manual p.2`. Each has a 25 × 18-bit signed multiplier, a 48-bit adder and pipeline registers `Background`. This fits our `acc += pixel × weight` exactly.

One DSP can do thousands of MACs

A MAC is one multiply-accumulate. Our network does 16,380 of them per image, but it needs only 2 DSPs, because each DSP does one MAC per cycle, one after another. This is called **time-sharing**. You trade time for hardware: fewer DSPs, more cycles.

Vivado may not use a DSP unless you ask

> **Important finding.** Our verification plan expects 2 DSPs (test S3). But our operands are small, so Vivado built the 9 × 8 multiplier from LUTs and used **no DSP at all**. The fix is one attribute.

Conv MAC (9 × 8, 24-bit accumulator)	DSP	LUTs	FFs	Slack at 100 MHz
Default	0	88	61	4.34 ns
With `use_dsp = "yes"`	1	0	4	7.96 ns

All numbers `Measured`. A plain 18 × 18 multiplier, for comparison, got a DSP without being asked. The FC MAC (8 × 8, 32-bit accumulator) also fits 1 DSP with the attribute, and `fc_engine` as written today used 0 DSPs and 248 LUTs.

```
(* use_dsp = "yes" *)
module mac ( ... );   // one line, before the module
```

Check the result in the utilization report under **DSPs**. With the attribute, the multiply and add both went inside the DSP, and so did most registers. That's why the LUT count dropped to 0.

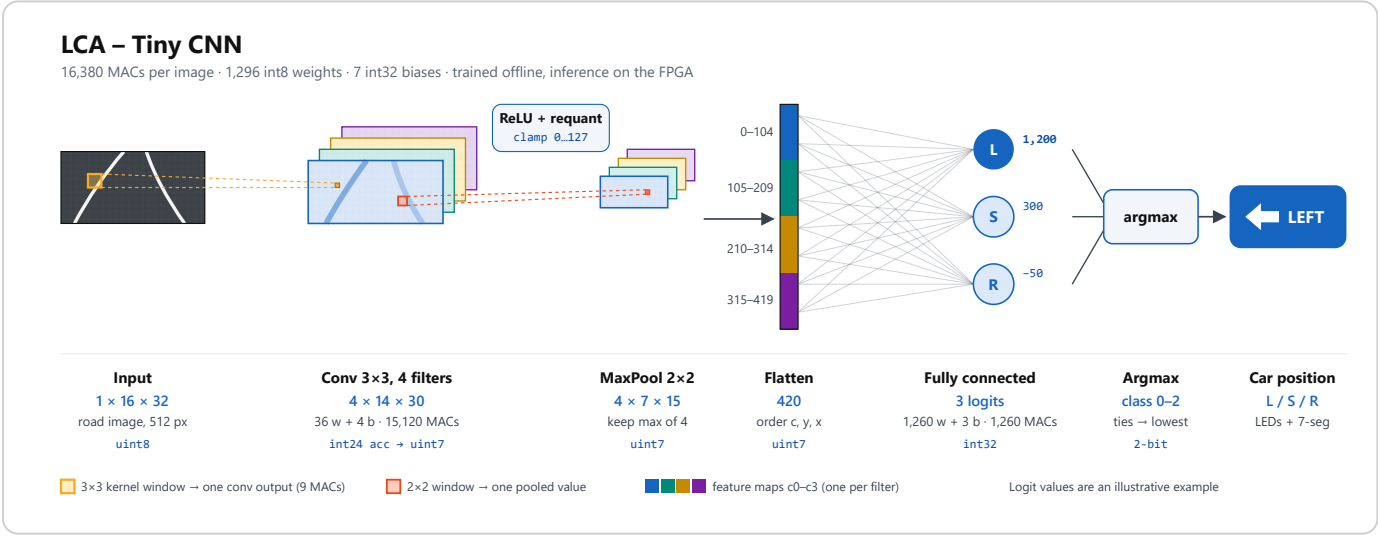
7. Choosing the right resource

You need	Use	Example in our design
Decisions, muxes, compares	LUT	`argmax3`, clamp in ReLU
Counters, state, registers	FF	`cycle_count`, FSM state
A few values, up to about 64 bytes	LUT ROM or RAM	`conv_w_rom`, biases
Hundreds of bytes or more	BRAM	Images, FC weights, buffers
Multiplying	DSP	Conv and FC MACs
Multiplying or dividing by a power of two	Just shift, no resource	`>>> S1`, `y << 5`

The 64-byte line is a project rule of thumb, not a hard limit.

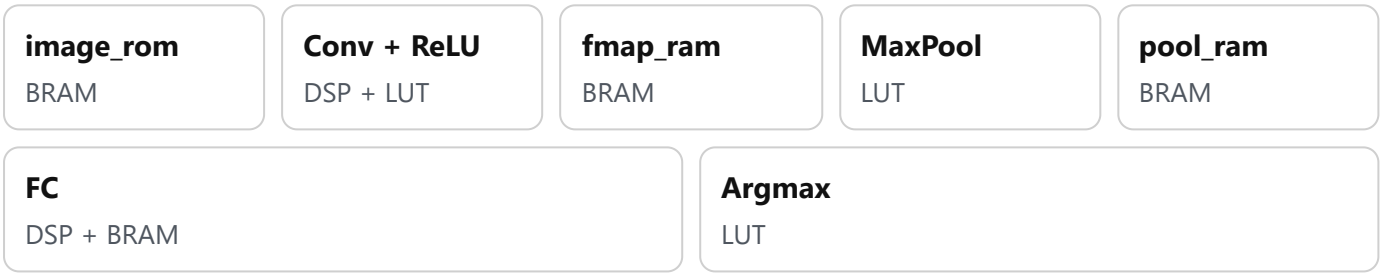
8. Our 1-layer design

Our LCA Tiny CNN looks at a 32 × 16 grayscale road image and picks LEFT, STRAIGHT or RIGHT. The name "1 layer" means it has one convolution layer.



Layer	Output shape	What it does
Input	`1×16×32`	512 uint8 pixels
Conv 3×3, 4 filters	`4×14×30`	Finds small patterns such as edges
ReLU + requant	`4×14×30`	Drops negatives, reduces to 7 bits
MaxPool 2×2	`4×7×15`	Halves the size, keeps strongest values
Fully connected	3 logits	Each class adds up 420 weighted values
Argmax	Class 0–2	Picks the highest score

The design runs **layer by layer**. Each stage reads from one memory and writes to the next, and a central FSM starts them in order. That's why the design needs buffers between layers, and why each stage can be tested alone.



9. Where each resource goes

Module	BRAM	DSP	LUT and FF
`image_rom`	2 RAMB36		
`conv_engine`		1	Address counters, control
`conv_w_rom`, `conv_b_rom`			LUT ROM
`relu_requant`			Shift and clamp
`fmap_ram`	1 RAMB18		
`maxpool_engine`			Address counters, compares
`pool_ram`	1 RAMB18		
`fc_engine`		1	Counters, 3 × 32-bit logits
`fc_w_rom`	1 RAMB18		
`fc_b_rom`, `label_rom`			LUT ROM
`argmax3`, `cnn_ctrl`			Compares, FSM, cycle counter
Board logic			Debounce, display, LEDs

10. How to calculate it

Everything starts from the shapes. Work in this order, because each step uses the one before.

Step 1: shapes

```
Conv output:  C_out × (H − K + 1) × (W − K + 1)
              4 × (16 − 3 + 1) × (32 − 3 + 1) = 4 × 14 × 30 = 1,680 values

Pool output:  C × H/2 × W/2
              4 × 7 × 15 = 420 values
```

Step 2: weights and biases

```
Conv weights: K² × C_in × C_out = 9 × 1 × 4 = 36
FC weights:   inputs × classes  = 420 × 3   = 1,260
Biases:       one per filter or class = 4 + 3 = 7
```

Step 3: memory in bytes

Memory	Calculation	Bytes
Images	16 × 32 × 16 × 1 B	8,192
Weights	1,296 × 1 B	1,296
Biases	7 × 4 B	28
Conv buffer	1,680 × 1 B	1,680
Pool buffer	420 × 1 B	420
Total		**11,616**

Step 4: BRAM blocks

Do this **per memory**, not on the total, because each memory takes its own block:

```
depth ≤ 2,048   →  1 RAMB18 (0.5 BRAM36)
depth ≤ 4,096   →  1 RAMB36
larger          →  ROUNDUP(depth / 4,096) RAMB36
```

Memory	Depth	Blocks	BRAM36	Evidence
`image_rom`	8,192	2 RAMB36	2.0	Measured
`fmap_ram`	1,680	1 RAMB18	0.5	Measured
`pool_ram`	420	1 RAMB18	0.5	Measured
`fc_w_rom`	1,260	1 RAMB18	0.5	Measured
Weights and biases in LUTs	36 + 7	None	0	Set by `ROM_STYLE`
Total			**3.5**	**3.5 of 50 = 7%**

Step 5: MACs and DSPs

```
Conv MACs: K² × C_in × output values = 9 × 1 × 1,680 = 15,120
FC MACs:   inputs × classes          = 420 × 3       =  1,260
Total                                                  16,380

DSPs = number of parallel MACs = 1 conv + 1 FC = 2
       2 of 90 = 2.2%
```

Step 6: cycles and time

Stage	Calculation	Cycles
Conv + ReLU	(9 taps + 3 pipeline) × 1,680	20,160
MaxPool	6 × 420	2,520
FC	3 × (420 + 4)	1,272
Control	About 10	10
Total		**23,962**

```
time = cycles / clock = 23,962 / 100,000,000 = 0.24 ms
```

The cycle formula is the plan's estimate. Test I4 replaces it with a real count.

Step 7: accumulator and address widths

```
Accumulator bits = product bits + ROUNDUP(log2(terms))
Conv: 17 + ROUNDUP(log2 9)   = 17 + 4 = 21 bits, we use 24
FC:   16 + ROUNDUP(log2 420) = 16 + 9 = 25 bits, we use 32

Address bits = ROUNDUP(log2(depth))
```

Memory	Depth	Address bits
`image_rom`	8,192	13
`fmap_ram`	1,680	11
`fc_w_rom`	1,260	11
`pool_ram`	420	9
`conv_w_rom`	36	6

Step 8: LUTs and FFs

You can't calculate these reliably by hand, because Vivado optimizes the logic. So measure them. These are out-of-context synthesis results for each module alone Measured :

Module	Source	LUTs	FFs
Board logic with stub	`top_basys3` report	57	136
`cnn_ctrl`	early version	26	27
`maxpool_engine`	early version	91	66
`fc_engine`, multiplier in LUTs	early version	248	153
`argmax3`	early version	50	0
Sum so far		**472**	**382**

This sum leaves out `conv_engine`, which isn't written yet. It's also only a guide: a report for the whole design can differ, because Vivado can share or trim logic across modules. Still, it's well on track for the plan's estimate of about 1,500 LUTs and 1,000 FFs.

---

11. Final budget

Resource	Available	Our design	Used	Our cap	Status
BRAM36	50	3.5	7%	20%	Calculated, blocks measured
DSP	90	2	2.2%	10%	Needs `use_dsp`
LUT	20,800	~1,500	~7%	25%	Plan estimate
FF	41,600	~1,000	~2.5%	10%	Plan estimate
Latency	30,000 cycles	23,962	80%	100%	Plan estimate

The caps are the project targets (D19). They're the team's own targets, not limits of the board.

---

12. What if we change it?

The same steps show what a change would cost. Figures use the plan's cycle formulas.

Change	BRAM36	DSP	Cycles	Why
Today	3.5	2	23,962	
32 test images	5.5	2	23,962	`image_rom` doubles to 4 RAMB36
8 filters in one layer	5.0	2	47,902	Everything after conv doubles
Second conv layer, 4 filters	4.5	3	33,622	Another engine and buffer

The second row shows why doing this on paper first is worthwhile: more stored images cost BRAM but no time. A bigger network costs time.

---

13. Try it yourself

Exercise 1: a memory with 3,000 × 8

```
2,048 < 3,000 ≤ 4,096, so 1 RAMB36 = 1.0 BRAM36
Address bits: ROUNDUP(log2 3,000) = 12
```

Exercise 2: change conv 1 to 6 filters. What changes?

```
Conv output: 6 × 14 × 30 = 2,520 values
Pool output: 6 × 7 × 15  = 630 values
Conv weights: 9 × 6 = 54      FC weights: 630 × 3 = 1,890
MACs: 9 × 2,520 + 1,890 = 24,570

fmap_ram 2,520 → 1 RAMB36 = 1.0   (past 2,048)
pool_ram 630   → 0.5
fc_w_rom 1,890 → 0.5
image_rom      → 2.0
BRAM total = 4.0 of 50

Cycles: 12 × 2,520 + 6 × 630 + 3 × (630 + 4) + 10 = 35,932
That breaks the 30,000 target.
```

Exercise 3: why not put all 1,296 weights in FFs?

```
1,296 × 8 = 10,368 FFs = 25% of the chip, plus large muxes to read them.
In BRAM, the 1,260 FC weights use 1 RAMB18 = 1% of the BRAM.
```

14. Cheat sheet

Remember	Rule
Limits	50 BRAM36, 90 DSP, 20,800 LUT, 41,600 FF, 100 MHz
BRAM size	2,048 bytes per RAMB18, 4,096 per RAMB36
BRAM cost	Per memory. Round up each one separately
BRAM read	1 cycle later
DSP count	Number of parallel MACs, not total MACs
DSP use	Add `use_dsp = "yes"` and check the report
FF count	Roughly 1 per registered bit
LUT count	Measure it with synthesis
Time	cycles ÷ clock frequency
Timing	WNS ≥ 0 at a 10 ns period

Sources: Digilent Basys 3 Reference Manual; AMD Artix 7 Data Sheet DS181 (v1.27.1); Vivado 2026.1 part data and out-of-context synthesis on `xc7a35tcpg236-1`; the project README. Measured 2026-11-02.